

SKILL 6

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

SOLUTION:-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("===== Mini Shell =====\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("MiniShell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        if (strcmp(input, "exit") == 0)
            break;

        int argc = 0;
        char *token = strtok(input, " ");

        while (token != NULL && argc < MAX_ARGS - 1)
        {
            args[argc++] = token;
            token = strtok(NULL, " ");
        }

        args[argc] = NULL;

        /* Display PATH variable */
        char *path = getenv("PATH");
        if (path != NULL)
        {
            printf("PATH = %s\n", path);
        }
        args[argc] = NULL;
```

```

char *path = getenv("PATH");
if (path != NULL)
{
    printf("PATH = %s\n", path);
}

pid_t pid = fork();

if (pid < 0)
{
    perror("Fork Failed");
    continue;
}

if (pid == 0)
{
    execvp(args[0], args);

    perror("Command Not Found");
    exit(EXIT_FAILURE);
}
else
{
    int status;

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child exited with status %d\n",
            WEXITSTATUS(status));
    }
    else
    {
        printf("Child terminated abnormally.\n");
        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child exited with status %d\n",
                WEXITSTATUS(status));
        }
        else
        {
            printf("Child terminated abnormally.\n");
        }
    }
}

printf("Shell Closed.\n");

return 0;
}

```

OUTPUT:-

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("===== Mini Shell =====\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {

```

Type 'exit' to quit.

```

MiniShell> ls -l
PATH = /usr/local/bin:/System/Cryptexes/App/usr/bin:/usr/bin:/bin:/usr/sbin:/sbin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/local/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/appleinternal/bin
total 600
-rwxr-xr-x  1 apple  staff   33360 Aug  5 12:00 skillq1
-rw-r--r--  1 apple  staff    815 Aug  5 12:01 skillq1.c
-rwxr-xr-x  1 apple  staff   33416 Aug  5 12:04 skillq2
-rw-r--r--  1 apple  staff   2265 Aug  5 12:04 skillq2.c
-rwxr-xr-x  1 apple  staff   34280 Aug 29 15:18 skillq3
-rw-r--r--  1 apple  staff   7701 Aug 29 15:18 skillq3.c
-rwxr-xr-x  1 apple  staff   33616 Aug  5 12:11 skillq4
-rw-r--r--  1 apple  staff   2748 Aug  5 12:09 skillq4.c
-rwxr-xr-x  1 apple  staff   33672 Aug  5 12:13 skillq5
-rw-r--r--  1 apple  staff   1715 Aug  5 12:13 skillq5.c
-rwxr-xr-x  1 apple  staff   33664 Aug  5 12:14 skillq6
-rw-r--r--  1 apple  staff   1692 Aug  5 12:14 skillq6.c
-rwxr-xr-x  1 apple  staff   50168 Aug  5 12:16 skillq7
-rw-r--r--  1 apple  staff   2064 Aug  5 12:16 skillq7.c
Child exited with status 0

```